4.3 Application of Forward Mode Automatic Differentiation

In [1]:
```
using CairoMakie
using Distributions
using Random
```

Define dual number and the corresponding operations

In [2]:
```
mutable struct Dual <: Number
    v::Float64 #value of the function
    ∂::Float64 #its derivative
end

import Base:+,-,*,sin,^,/,cos
+(a::Dual,b::Dual)=Dual(a.v+b.v,a.∂+b.∂)
+(a::Dual,b::Number)=Dual(a.v+b,a.∂)
+(a::Number,b::Dual)=Dual(a+b.v,b.∂)
-(a::Dual,b::Dual)=Dual(a.v-b.v,a.∂-b.∂)
-(a::Number,b::Dual)=Dual(a-b.v,-b.∂)
-(a::Dual,b::Number)=Dual(a.v-b,a.∂)
*(a::Dual,b::Dual)=Dual(a.v*b.v,a.v*b.∂+b.v*a.∂)
*(a::Number,b::Dual)=Dual(a*b.v,a*b.∂)
/(a::Dual,b::Dual)=Dual(a.v/b.v,(b.v*a.∂-a.v*b.∂)/(b.v)^2)
/(a::Number,b::Dual)=Dual(a/b.v,(-a*b.∂)/(b.v)^2)
#*(a::Float64,b::Dual)=Dual(a*b.v,a*b.∂)
^(a::Dual,n::Integer)=Dual(a.v^n,n*a.v^(n-1)*a.∂)
sin(a::Dual)=Dual(sin(a.v),cos(a.v)*a.∂)
cos(a::Dual)=Dual(cos(a.v),-sin(a.v)*a.∂)
```

```
cos (generic function with 26 methods)
```

Now let's do an example where we have to use dual numbers to find derivatives. We will use the same example as we have always done, where we want to find the minimum material needed to make a cylinder to contain 1m^3 of water.

$$1 = \pi r^2 h$$

$$A(r) = \pi r^2 + \pi r^2 + 2\pi r h$$

$$A(r) = 2\pi r^2 + \frac{2}{r}$$

$$f(r) = dA(r)/dr = 4\pi r - \frac{2}{r^2} = 0$$

We want to find r such that $f(r) = 0$. If we use Newton-Raphson technique

$$r_{i+1} = r_i - f(r_i)/dfdr(r_i).$$

Hence need to calculate df/dr. So far we have calculated the derivative by hand and by using `ForwardDiff` / `Zygote`. Now, we will calculate df/dr using Forward Mode Automatic Differentiation and Dual numbers.

Define a function $f(r)$

In [3]:
```
f(r)=4π*r-2.0(1/r)*(1/r)
```

```
f (generic function with 1 method)
```

In [4]:
```
f(Dual(0.2,1.0))
```

```
Dual(-47.486725877128166, 512.5663706143591)
```

Code to find r where $f(r) = 0$ using the Newton Raphson Method. In the code below, we obtain the derivative using Forward Mode Automatic Differentiation via Dual numbers. The starting guess is $r = 1$.

In [5]:
```
r=Dual(1.0,1.0)

for i=1:50
        println(r)
        r=r-f(r).v/f(r).∂
end
```

```
Dual(1.0, 1.0)
Dual(0.3621795105078358, 1.0)
Dual(0.4727153376004709, 1.0)
Dual(0.5323948682518493, 1.0)
Dual(0.5417564911696957, 1.0)
Dual(0.5419260170637374, 1.0)
Dual(0.5419260701392838, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
Dual(0.5419260701392891, 1.0)
Dual(0.541926070139289, 1.0)
```

Dual Number Example with Multivariate functions

We will now use our own Forward Mode Automatic differentiation (using Dual numbers) code to find the derivatives needed to solve the following set of two nonlinear equations.

$$f_1(\vec{x}) = x_1^2 - x_2 + 1 = 0 \quad (1)$$
$$f_2(\vec{x}) = 3\cos(x_1) - x_2 = 0 \quad (2)$$

We have to iterate using the following formula

$$\begin{bmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{bmatrix} \begin{Bmatrix} x_1^{(i+1)} - x_1^{(i)} \\ x_2^{(i+1)} - x_2^{(i)} \end{Bmatrix} = - \begin{Bmatrix} f_1(x_1^{(i)}, x_2^{(i)}) \\ f_2(x_1^{(i)}, x_2^{(i)}) \end{Bmatrix}$$

Define the vector function

In [6]:
```
VectorFunc(x)=[x[1]^2-x[2]+1,
               3*cos(x[1])-x[2]]
```

```
VectorFunc (generic function with 1 method)
```

The derivative with respect to $\partial f_1/\partial x_1$ at point $(x_1, x_2) = (1, 1)$ can be calculated as

In [7]:
```
df1dx1=VectorFunc([Dual(1.0,1.0),Dual(1.0,0.0)])[1].∂
```

```
2.0
```

The derivative with respect to $\partial f_2/\partial x_1$ at point $(x_1, x_2) = (1, 1)$ can be calculated as

In [8]:
```
df1dx1=VectorFunc([Dual(1.0,1.0),Dual(1.0,0.0)])[2].∂
```

```
-2.5244129544236893
```

To calculate the Jacobian

$$\begin{bmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{bmatrix}$$

at point $(x_1, x_2) = (1, 1)$ you can use the code below

In [9]:
```
xvals=[1,1]
J=hcat(getfield.(VectorFunc([Dual(xvals[1],1.0),Dual(xvals[2],0.0)])[:],:
getfield.(VectorFunc([Dual(xvals[1],0.0),Dual(xvals[1],1.0)])[:],:∂))
```

```
2×2 Matrix{Float64}:
  2.0      -1.0
 -2.52441  -1.0
```

Now we iterate using Newton-Raphson to solve for x_1 and x_2.

In [10]:
```
xvals=[1,1]
for i=1:10
    F=VectorFunc(xvals)
    J=hcat(getfield.(VectorFunc([Dual(xvals[1],1.0),Dual(xvals[2],0.0)])[
getfield.(VectorFunc([Dual(xvals[1],0.0),Dual(xvals[1],1.0)])[:],:∂))
```

```
    delta=-J\F
    xvals+=delta
    println(xvals)
end
```

```
[0.9162116530444182, 1.8324233060888364]
[0.9131363112988626, 1.8338084652856412]
[0.9131320007943952, 1.8338100508561948]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
```

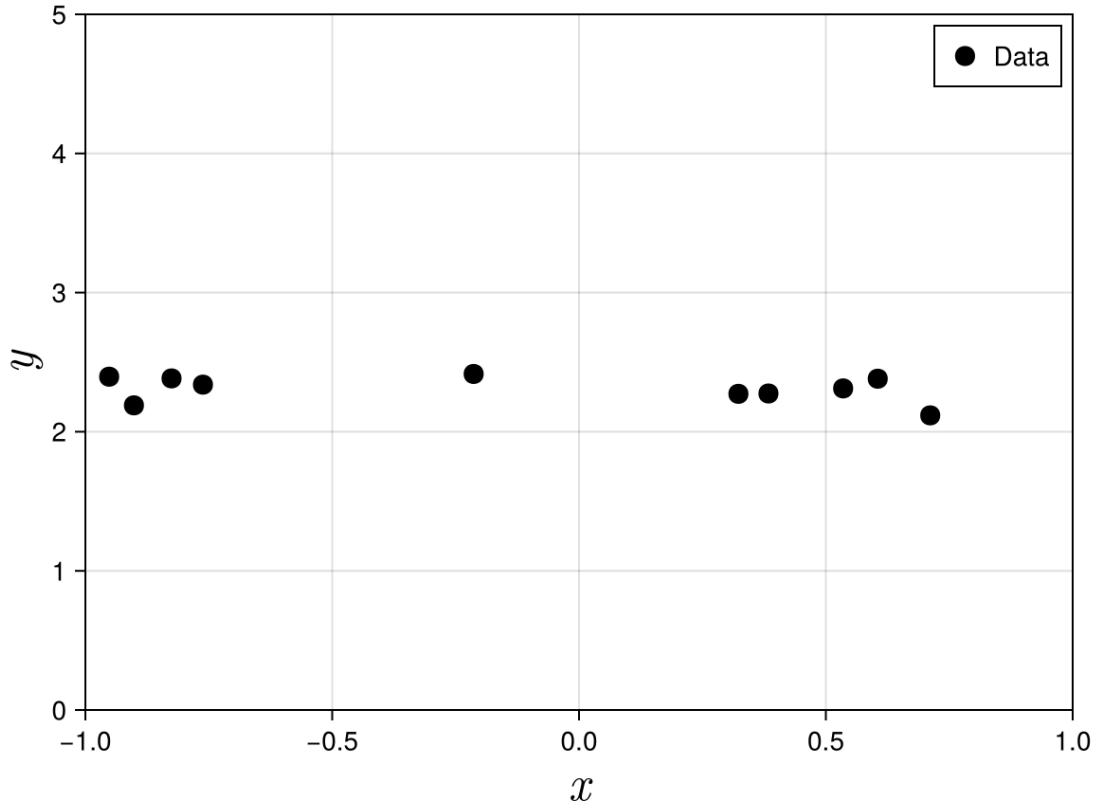
Let's see how we can use our own Dual mode Forward Mode automatic differentiation to solve a very easy regression problem. We have a constant data set and we want to use

$f_{model} = a_0$ to approximate the data

In [11]:
```
N_SAMPLES=10
rng=Xoshiro(1) #my random number generator

xi=rand(rng,Uniform(-1,1),N_SAMPLES) #generate random values of x between
yi=rand(rng,Normal(2.3,0.1),N_SAMPLES)

fig = Figure()
ax = Axis(fig[1, 1], xlabel = L"x", ylabel = L"y",
    xlabelsize=25,
    ylabelsize=25,
    limits=(-1,1,0,5),
    backgroundcolor = :transparent)
scatter!(ax,xi,yi;label="Data",color=:black,markersize=15)
axislegend(ax;position = :rt)
save("../figures/04p03ApplicationOfForwardMode.svg",fig)
display(fig)
```




```
CairoMakie.Screen{IMAGE}
```

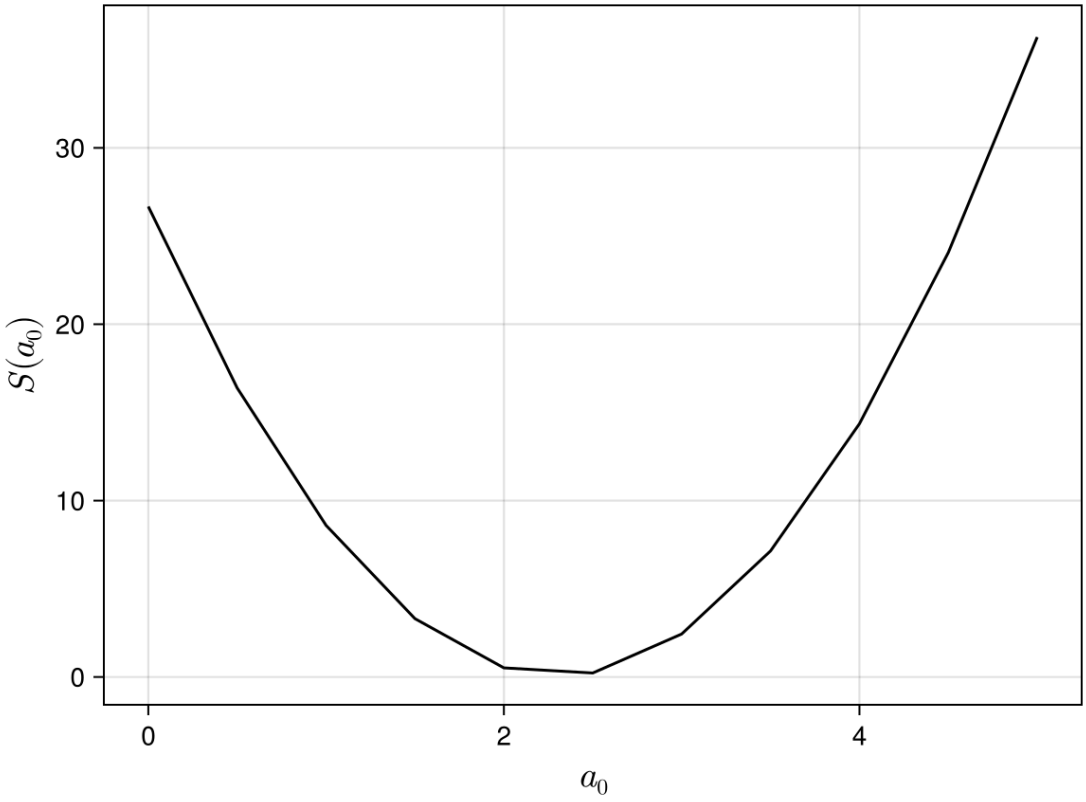
Define the loss function $S(a_0) = \frac{1}{2}\sum_{i=1}^{N}(y_i - a_0)^2$

In [12]:
```
S(a0)=0.5*sum((yi.-a0).^2)
```

```
S (generic function with 1 method)
```

For this problem, it is easy to plot and visualise how S changes with a_0. Let's plot $S(a_0)$ for $a_0 \in [0, 5]$

In [13]:
```
fig = Figure()
a0vec=0:0.5:5
ax = Axis(fig[1, 1], xlabel = L"a_0", ylabel = L"S(a_0)",
    xlabelsize=20,
    ylabelsize=20,
    backgroundcolor = :transparent)
lines!(ax,a0vec,S.(a0vec),color=:black)
save("../figures/04p03ApplicationOfForwardMode02.svg",fig)
display(fig)
```




```
CairoMakie.Screen{IMAGE}
```

Check to see if I can calculate derivative at $a_0 = 1$

In [14]:
```
a0=Dual(1.0,1.0)
S(a0).∂
```

```
-13.079059040647497
```

and at $a_0 = 4$

In [15]:
```
a0=Dual(4.0,1.0)
S(a0).∂
```

```
16.920940959352503
```

The derivative of S at $a_0 = 1$ is negative and $a_0 = 4$ is positive. This is consistent with the graph above, giving us confidence that everything is working as it should be.

Now we will use gradient descent to find the location of the minimum, starting with $a_0 = 1.0$.

In [16]:
```
a0=Dual(1.0,1.0)

α=0.01

for i=1:500
    a0.v=a0.v-α*S(a0).∂
    println(a0.v)
end
```

```
1.130790590406475
1.2485021217723025
1.3544425000015472
1.4497888404078674
1.5356005467735556
1.612831082502675
1.6823385646588824
1.7448952985994692
1.8011963591459972
1.8518673136378725
1.8974711726805602
1.9385146458189793
1.9754537716435563
2.0086989848856756
2.038619676803583
2.0655482995297
2.089784059983205
2.1115962443913596
2.1312272103586984
2.1488950797293036
2.1647961621628484
2.1791071363530388
2.19198701312421
2.203578902218264
2.2140116024029126
2.2234010325690963
2.2318515197186617
2.2394569581532706
2.2463018527444185
2.2524622578764517
2.2580066224952815
2.2629965506522285
2.2674874859934806
2.2715293278006077
2.275166985427022
2.2784408772907945
2.28138737996819
2.284039232377846
2.2864258995465363
2.2885738999983576
2.2905071004049966
2.292246980770972
2.2938128731003498
2.29522217619679
2.296490548983586
2.2976320844917026
2.298659466449007
2.2995841102105814
2.300416289595998
2.3011652510428733
2.301839316345061
2.30244597511703
2.302991968011802
2.3034833616170967
2.303925615861862
2.304323644682151
2.304681870620411
2.305004273964845
2.3052944369748354
2.3055555836838266
```

```
2.305790615721919
2.306002144556202
2.306192520507057
2.306363858862826
2.3065180633830185
2.3066568474511917
2.3067817531125474
2.3068941682077675
2.3069953417934657
2.3070863980205942
2.3071683486250096
2.3072421041689837
2.3073084841585603
2.3073682261491792
2.307421993940736
2.3074703849531373
2.3075139368642987
2.307553133584344
2.3075884106323845
2.307620159975621
2.307648734384534
2.3076744513525558
2.3076975966237754
2.307718427367873
2.307737175037561
2.30775404794028
2.307769233552727
2.307782900603929
2.3077952009500113
2.307806271261485
2.3078162345418116
2.3078252014941056
2.30783327175117
2.307840534982528
2.3078470718907504
2.3078529551081504
2.3078582500038105
2.3078630154099042
2.3078673042753888
2.307871164254325
2.307874638235367
2.3078777648183055
2.30788057874295
2.30788311127513
2.307885390554092
2.307887441905158
2.307889288121117
2.3078909497154805
2.3078924451504075
2.307893791041842
2.3078950023441327
2.307896092516194
2.30789707367105
2.30789795671042
2.307898751445853
2.3078994667077426
2.3079001104434433
2.307900689805574
2.3079012112314916
2.3079016805148176
```

```
2.307902102869811
2.307902482989305
2.3079028250968494
2.3079031329936393
2.3079034101007503
2.3079036594971503
2.30790388395391
2.307904085964994
2.3079042677749695
2.3079044314039474
2.3079045786700276
2.3079047112095
2.307904830495025
2.3079049378519976
2.307905034473273
2.307905121432421
2.3079051996956537
2.3079052701325633
2.307905333525782
2.3079053905796787
2.307905441928186
2.3079054881418424
2.3079055297341333
2.307905567167195
2.3079056008569503
2.3079056311777304
2.3079056584664324
2.3079056830262643
2.307905705130113
2.3079057250235766
2.307905742927694
2.3079057590413994
2.3079057735437343
2.307905786595836
2.3079057983427274
2.3079058089149296
2.3079058184299117
2.3079058269933954
2.307905834700531
2.307905841636953
2.3079058478797325
2.307905853498234
2.3079058585548857
2.307905863105872
2.3079058672017596
2.3079058708880584
2.3079058742057277
2.30790587719163
2.307905879878942
2.307905882297523
2.3079058844742457
2.307905886433296
2.307905888196441
2.307905889783272
2.3079058912114196
2.307905892496753
2.3079058936535524
2.307905894694672
2.30790589563168
2.307905896474987
```

```
2.3079058972339634
2.307905897917042
2.307905898531813
2.3079058990851067
2.307905899583071
2.3079059000312387
2.30790590043459
2.3079059007976057
2.30790590112432
2.307905901418363
2.3079059016830015
2.3079059019211763
2.3079059021355337
2.3079059023284554
2.307905902502085
2.3079059026583515
2.3079059027989914
2.307905902925567
2.3079059030394853
2.3079059031420117
2.3079059032342855
2.307905903317332
2.307905903392074
2.3079059034593414
2.3079059035198823
2.307905903574369
2.307905903623407
2.3079059036675416
2.3079059037072622
2.307905903743011
2.307905903775185
2.307905903804141
2.307905903830202
2.3079059038536567
2.307905903874766
2.3079059038937646
2.307905903910863
2.3079059039262515
2.3079059039401013
2.307905903952566
2.3079059039637846
2.307905903973881
2.307905903982968
2.3079059039911463
2.3079059039985066
2.307905904005131
2.307905904011093
2.3079059040164585
2.3079059040212875
2.307905904025634
2.3079059040295453
2.3079059040330656
2.307905904036234
2.3079059040390857
2.307905904041652
2.307905904043962
2.3079059040460406
2.3079059040479115
2.3079059040495955
2.3079059040511107
```

```
2.3079059040524745
2.307905904053702
2.307905904054807
2.307905904055801
2.307905904056696
2.3079059040575016
2.3079059040582264
2.3079059040588787
2.307905904059466
2.3079059040599943
2.30790590406047
2.307905904060898
2.307905904061283
2.30790590406163
2.307905904061942
2.3079059040622227
2.3079059040624754
2.307905904062703
2.3079059040629075
2.307905904063092
2.3079059040632575
2.3079059040634067
2.307905904063541
2.3079059040636616
2.3079059040637704
2.3079059040638685
2.3079059040639565
2.307905904064036
2.3079059040641074
2.307905904064172
2.3079059040642296
2.3079059040642815
2.307905904064328
2.3079059040643704
2.307905904064408
2.3079059040644423
2.307905904064473
2.3079059040645005
2.3079059040645253
2.307905904064548
2.307905904064568
2.307905904064586
2.3079059040646026
2.3079059040646173
2.3079059040646306
2.3079059040646426
2.3079059040646532
2.307905904064663
2.307905904064672
2.30790590406468
2.307905904064687
2.307905904064693
2.307905904064699
2.307905904064704
2.3079059040647083
2.3079059040647123
2.307905904064716
2.3079059040647194
2.3079059040647225
2.307905904064725
```

```
2.307905904064728
2.30790590406473
2.307905904064732
2.3079059040647336
2.3079059040647354
2.3079059040647367
2.307905904064738
2.3079059040647394
2.3079059040647403
2.307905904064741
2.307905904064742
2.307905904064743
2.307905904064744
2.3079059040647443
2.3079059040647447
2.307905904064745
2.3079059040647456
2.307905904064746
2.3079059040647465
2.307905904064747
2.3079059040647474
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
```

```
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
```

```
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
```

```
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
2.307905904064748
```

a_0 convergest to a value close to 2.3 which is what we would expect.

In [17]: